

```

In [14]: # =====
# DECODING STEAM:
# A DATA-DRIVEN ANALYSIS OF 6.4 MILLION USER REVIEWS
# =====
# Focus:
# Data Quality Audit, Market Concentration,
# Review Sentiment, and Helpful-Review Engagement
# =====

# =====
# LIBRARIES
# =====

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

from matplotlib.ticker import FuncFormatter
from IPython.display import display, Markdown

# =====
# PHASE 1: CONFIGURATION
# =====

sns.set_theme(style="whitegrid")

plt.rcParams["figure.figsize"] = (14, 8)
plt.rcParams["font.size"] = 12

pd.set_option("display.max_columns", 20)
pd.set_option("display.max_colwidth", 80)
pd.set_option("display.float_format", lambda x: f"{x:,.2f}")

# Format large values as Millions (M) and Thousands (K)
def business_formatter(x, pos):

    if abs(x) >= 1_000_000:
        return f"{x / 1_000_000:.1f}M"

    if abs(x) >= 1_000:
        return f"{x / 1_000:.0f}K"

    return f"{x:.0f}"

formatter = FuncFormatter(business_formatter)

# Describe correlation strength and direction
def describe_correlation(value):

    absolute_value = abs(value)

    if absolute_value < 0.10:

```

```

        strength = "very weak"

    elif absolute_value < 0.30:
        strength = "weak"

    elif absolute_value < 0.50:
        strength = "moderate"

    else:
        strength = "strong"

    if value > 0:
        direction = "positive"

    elif value < 0:
        direction = "negative"

    else:
        direction = "zero"

    return strength, direction

display(Markdown("""
# Decoding Steam: A Data-Driven Analysis of 6.4 Million User Reviews

### Analysis of review sentiment, market concentration, approval rates, a

**Tools:** Python, Pandas, NumPy, Matplotlib, and Seaborn
**Focus:** Data quality, review activity, user approval, and helpful-revi
"""))

# =====
# PHASE 2: DATA INGESTION
# =====

print("Step 1: Loading the Steam dataset...")

df = pd.read_csv(
    r"C:\Users\שלי\מחשב\Downloads\archive (3)\dataset.csv",
    low_memory=False
)

# Standardize column names
df.columns = (
    df.columns
    .str.strip()
    .str.lower()
    .str.replace(" ", "_")
)

# =====
# DATASET VALIDATION
# =====

required_columns = {
    "app_id",

```

```

        "app_name",
        "review_score",
        "review_votes"
    }

    missing_columns = required_columns - set(df.columns)

    if missing_columns:

        raise ValueError(
            "The selected file is not the required Steam dataset.\n\n"
            f"Missing columns: {sorted(missing_columns)}\n\n"
            f"Available columns: {df.columns.tolist()}\n\n"
            "The Steam dataset should contain app_id, app_name, "
            "review_score, and review_votes."
        )

    print("Steam dataset validated successfully.")

    # Convert analytical fields to numeric values
    df["review_score"] = pd.to_numeric(
        df["review_score"],
        errors="coerce"
    )

    df["review_votes"] = pd.to_numeric(
        df["review_votes"],
        errors="coerce"
    )

    # =====
    # DATA QUALITY AUDIT
    # =====

    print("\n" + "=" * 50)
    print("DATA QUALITY AUDIT REPORT")
    print("=" * 50)

    # 1. Basic information
    print(f"Total Records: {len(df):,}")
    print(f"Total Columns: {len(df.columns):,}")

    # 2. Missing values analysis
    print("\n--- Missing Values per Column ---")

    null_counts = df.isnull().sum()

    print(
        null_counts[null_counts > 0]
        if null_counts.sum() > 0
        else "No missing values found."
    )

```

```

# 3. Zero values analysis
print("\n--- Zero Values Analysis ---")

numeric_columns = df.select_dtypes(
    include=[np.number]
).columns

for column in numeric_columns:

    zero_count = (df[column] == 0).sum()

    zero_percentage = (
        zero_count / len(df) * 100
    )

    print(
        f"{column}: {zero_count:,} zeros "
        f"({zero_percentage:.2f}%)"
    )

# 4. Review-votes validation
review_vote_values = sorted(
    df["review_votes"]
    .dropna()
    .unique()
    .tolist()
)

print("\n--- Review Votes Validation ---")
print(f"Unique review_votes values: {review_vote_values}")

if set(review_vote_values).issubset({0, 1}):

    print("Confirmed: review_votes is a binary field.")
    print("0 = the review was not marked as helpful.")
    print("1 = the review was marked as helpful.")

else:

    print(
        "review_votes contains values above 1. "
        "Every value above 0 will be treated as a review "
        "that received Helpful engagement."
    )

# =====
# DATA CLEANING
# =====

initial_records = len(df)

df_clean = df.dropna(
    subset=[
        "app_name",

```

```

        "review_score",
        "review_votes"
    ]
).copy()

# Keep only valid review scores
df_clean = df_clean[
    df_clean["review_score"].isin([-1, 1])
].copy()

# Create a binary Helpful indicator
df_clean["is_helpful"] = (
    df_clean["review_votes"] > 0
).astype(int)

removed_records = (
    initial_records - len(df_clean)
)

print("\n" + "=" * 50)
print("DATA CLEANING SUMMARY")
print("=" * 50)

print(f"Original Records: {initial_records:,}")
print(f"Removed Records: {removed_records:,}")
print(f"Clean Review Records: {len(df_clean):,}")

# Helpful-review distribution after cleaning
not_helpful_percentage = (
    df_clean["is_helpful"].eq(0).mean()
    * 100
)

helpful_percentage = (
    df_clean["is_helpful"].eq(1).mean()
    * 100
)

print("\n--- Helpful-Review Distribution ---")

print(
    f"Reviews not marked as helpful: "
    f"{not_helpful_percentage:.2f}%"
)

print(
    f"Reviews marked as helpful: "
    f"{helpful_percentage:.2f}%"
)

display(Markdown(f"""
### Data Cleaning Explanation

```

- The original dataset contained ****{initial_records:,} individual review**
- ****{removed_records:,} records**** were removed because essential fields w
- The cleaned dataset contains ****{len(df_clean):,} valid reviews****.
- ****{not_helpful_percentage:.2f}%**** of valid reviews were not marked as h
- Zero values were retained because ``0`` represents a valid review that di

The percentage of zero values changed slightly after cleaning because row
 """))

```
# =====
# PREVIEW BEFORE AGGREGATION
# =====
```

```
display(Markdown("""
## Review-Level Data Preview
```

Before aggregation, each row represents ****one individual Steam review****.

The same game can appear in many different rows because one game may have
 """))

```
review_preview_columns = [
    "app_id",
    "app_name"
]
```

```
if "review_text" in df_clean.columns:
    review_preview_columns.append(
        "review_text"
    )
```

```
review_preview_columns.extend([
    "review_score",
    "review_votes",
    "is_helpful"
])
```

```
display(
    df_clean[
        review_preview_columns
    ].head(10)
)
```

```
# =====
# MASTER AGGREGATION
# =====
```

```
game_stats = (
    df_clean
    .groupby("app_name")
    .agg(
        total_reviews=(
            "review_score",
```

```

        "count"
    ),
    avg_score=(
        "review_score",
        "mean"
    ),
    helpful_reviews=(
        "is_helpful",
        "sum"
    )
)
.reset_index()
)

# Convert average review score from -1 / 1
# into an approval percentage
game_stats["approval_rate"] = (
    (game_stats["avg_score"] + 1)
    / 2
    * 100
)

# Calculate the percentage of reviews
# marked as Helpful for each game
game_stats["helpful_review_rate_pct"] = (
    game_stats["helpful_reviews"]
    / game_stats["total_reviews"]
    * 100
)

# Reorder the columns for a clearer table
game_stats = game_stats[
    [
        "app_name",
        "total_reviews",
        "approval_rate",
        "helpful_reviews",
        "helpful_review_rate_pct"
    ]
]

# =====
# EXPLANATION OF THE 9,363 ROWS
# =====

clean_review_count = len(df_clean)

unique_game_count = (
    df_clean["app_name"].nunique()
)

game_stats_count = len(game_stats)

```

```

print("\n" + "=" * 50)
print("REVIEW-LEVEL TO GAME-LEVEL AGGREGATION")
print("=" * 50)

print(
    f"Clean individual review rows: "
    f"{clean_review_count:,}"
)

print(
    f"Unique game names: "
    f"{unique_game_count:,}"
)

print(
    f"Rows in the aggregated game_stats table: "
    f"{game_stats_count:,}"
)

display(Markdown(f"""
## From Individual Reviews to Game-Level Statistics

The cleaned dataset contains {clean_review_count:,} rows, where each
The command `df_clean.groupby("app_name")` groups together all reviews be
After the aggregation:

- Each row in `game_stats` represents one game, not one review.
- The resulting table contains {game_stats_count:,} game-level rows.
- In this dataset, this produces 9,363 unique game-name summaries.
- The original reviews were not deleted. They were summarized into game-l

Each game-level row contains:

- `total_reviews` – total number of reviews for the game
- `approval_rate` – percentage of positive reviews
- `helpful_reviews` – number of reviews marked as Helpful
- `helpful_review_rate_pct` – percentage of reviews marked as Helpful
"""))

# Confirm that one row was created
# for each unique game name
assert game_stats_count == unique_game_count

# =====
# PREVIEW AFTER AGGREGATION
# =====

display(Markdown("""
## Game-Level Table Preview

After aggregation, each row represents one game.

The following preview shows how multiple individual reviews were summariz
"""))

```

```

display(
    game_stats.head(10)
)

print("\n--- Aggregated Game-Level Statistics ---")

display(
    game_stats[
        [
            "total_reviews",
            "approval_rate",
            "helpful_reviews",
            "helpful_review_rate_pct"
        ]
    ]
    .describe()
    .round(2)
)

print("=" * 50 + "\n")

# =====
# PHASE 3: BUSINESS ANALYSIS
# =====

# =====
# Q1: GLOBAL SENTIMENT BENCHMARK
# =====

sentiment_counts = (
    df_clean["review_score"]
    .value_counts()
    .reindex(
        [1, -1],
        fill_value=0
    )
)

positive_rate = (
    sentiment_counts.loc[1]
    / sentiment_counts.sum()
    * 100
)

plt.figure(figsize=(8, 6))

plt.pie(
    sentiment_counts.values,
    labels=[
        "Positive",
        "Negative"
    ],
    autopct="%1.1f%%",

```

```

        colors=[
            "#2ecc71",
            "#e74c3c"
        ],
        startangle=90,
        explode=(0, 0.05)
    )

plt.title(
    "Q1: Global Market Sentiment Benchmark",
    fontsize=16,
    fontweight="bold"
)

plt.show()

display(Markdown(f"""
### Q1 FINDINGS & BUSINESS INSIGHTS

* **Data Point:** Positive reviews account for **{positive_rate:.1f}% of
* **Interpretation:** Overall review sentiment is strongly positive, indi
* **Business Relevance:** Games performing substantially below this bench
"""))

# =====
# Q2: MARKET CONCENTRATION – PARETO ANALYSIS
# =====

pareto_df = (
    game_stats
    .sort_values(
        "total_reviews",
        ascending=False
    )
    .copy()
)

pareto_df["cumulative_review_pct"] = (
    pareto_df["total_reviews"]
    .cumsum()
    / pareto_df["total_reviews"].sum()
    * 100
)

pareto_df["catalog_pct"] = (
    np.arange(
        1,
        len(pareto_df) + 1
    )
    / len(pareto_df)
    * 100
)

top_five_count = max(
    1,

```

```

        int(
            np.ceil(
                len(pareto_df) * 0.05
            )
        )
    )

top_five_share = (
    pareto_df
    .head(top_five_count)["total_reviews"]
    .sum()
    / pareto_df["total_reviews"].sum()
    * 100
)

plt.figure()

plt.plot(
    pareto_df["catalog_pct"],
    pareto_df["cumulative_review_pct"],
    color="navy",
    linewidth=3
)

plt.axhline(
    y=80,
    color="red",
    linestyle="--",
    label="80% Review Activity Line"
)

plt.axvline(
    x=5,
    color="gray",
    linestyle="--",
    label="Top 5% of Games"
)

plt.title(
    "Q2: Market Concentration – Pareto Analysis",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("% of Total Game Catalog")
plt.ylabel("Cumulative % of Total Reviews")
plt.legend()
plt.show()

display(Markdown(f"""
Q2 FINDINGS & BUSINESS INSIGHTS

* **Data Point:** The top 5% of games generated approximately {top_five}
* **Interpretation:** Review activity is highly concentrated among a rela
* **Business Relevance:** Games outside the leading group may face greate
"""))

```

```

# =====
# Q3: POLARIZED MAJOR TITLES
# =====

major_titles = game_stats[
    game_stats["total_reviews"] > 20_000
].copy()

major_titles["distance_from_50"] = abs(
    major_titles["approval_rate"] - 50
)

top_polarized_games = (
    major_titles
    .nsmallest(
        10,
        "distance_from_50"
    )
)

plt.figure()

sns.barplot(
    data=top_polarized_games,
    x="approval_rate",
    y="app_name",
    hue="app_name",
    palette="coolwarm",
    legend=False
)

plt.axvline(
    x=50,
    color="black",
    linestyle="--",
    label="50/50 Approval Split"
)

plt.title(
    "Q3: Most Polarized Major Titles (>20K Reviews)",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Approval Rate (%)")
plt.ylabel("Game")
plt.legend()
plt.show()

display(Markdown("""
### Q3 FINDINGS & BUSINESS INSIGHTS

* **Data Point:** Among games with more than 20K reviews, these titles re
* **Interpretation:** The results indicate highly divided user sentiment.
* **Next Step:** Review-text or topic analysis would be required to ident

```

```

"""))

# =====
# Q4: GAME PORTFOLIO POSITIONING
# =====

matrix_df = game_stats[
    game_stats["total_reviews"] > 5_000
].copy()

fig, ax = plt.subplots()

sns.scatterplot(
    data=matrix_df,
    x="approval_rate",
    y="total_reviews",
    size="helpful_reviews",
    sizes=(20, 1000),
    hue="approval_rate",
    palette="RdYlGn",
    alpha=0.6,
    ax=ax
)

ax.set_yscale("log")

ax.yaxis.set_major_formatter(
    formatter
)

ax.axvline(
    x=80,
    color="black",
    linestyle="--",
    alpha=0.4
)

plt.title(
    "Q4: Game Portfolio Positioning – Approval Rate vs. Review Volume",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Approval Rate (%)")
plt.ylabel("Total Reviews")
plt.show()

display(Markdown("""
### Q4 FINDINGS & BUSINESS INSIGHTS

* **Data Point:** Several games with high review volumes recorded approval rates between 70% and 80%.
* **Interpretation:** These games combine broad review activity with competitive approval rates.
* **Business Relevance:** High-volume, lower-approval games may be prioritized for targeted marketing or quality improvement efforts.
"""))

```

```

    """))

# =====
# Q5: HELPFUL REVIEW RATE BY SENTIMENT
# =====

helpful_by_sentiment = (
    df_clean
    .groupby("review_score")["is_helpful"]
    .mean()
    .mul(100)
    .reset_index(
        name="helpful_review_rate_pct"
    )
)

helpful_by_sentiment["sentiment"] = (
    helpful_by_sentiment["review_score"]
    .map({
        1: "Positive",
        -1: "Negative"
    })
)

negative_helpful_rate = (
    helpful_by_sentiment.loc[
        helpful_by_sentiment["sentiment"]
        == "Negative",
        "helpful_review_rate_pct"
    ]
    .iloc[0]
)

positive_helpful_rate = (
    helpful_by_sentiment.loc[
        helpful_by_sentiment["sentiment"]
        == "Positive",
        "helpful_review_rate_pct"
    ]
    .iloc[0]
)

helpful_rate_ratio = (
    negative_helpful_rate
    / positive_helpful_rate
)

plt.figure()

sns.barplot(
    data=helpful_by_sentiment,
    x="sentiment",
    y="helpful_review_rate_pct",
    order=[

```

```

        "Negative",
        "Positive"
    ],
    hue="sentiment",
    palette={
        "Negative": "#e74c3c",
        "Positive": "#3498db"
    },
    legend=False
)

plt.title(
    "Q5: Share of Reviews Marked as Helpful by Sentiment",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Review Sentiment")
plt.ylabel("Reviews Marked as Helpful (%)")
plt.show()

display(Markdown(f"""
### Q5 FINDINGS & BUSINESS INSIGHTS

* **Data Point:** **{negative_helpful_rate:.1f}% of negative reviews** we
* **Interpretation:** Negative reviews were approximately **{helpful_rate}
* **Business Relevance:** Critical feedback may receive greater community
"""))

# =====
# Q6: APPROVAL RATE BY REVIEW-VOLUME TIER
# =====

review_volume_bins = [
    0,
    1_000,
    10_000,
    100_000
]

review_volume_labels = [
    "Low Volume (<1K)",
    "Medium Volume (1K-10K)",
    "High Volume (10K-100K)"
]

game_stats["review_volume_tier"] = pd.cut(
    game_stats["total_reviews"],
    bins=review_volume_bins,
    labels=review_volume_labels
)

tier_data = (
    game_stats
    .dropna(

```

```

        subset=["review_volume_tier"]
    )
    .groupby(
        "review_volume_tier",
        observed=False
    )["approval_rate"]
    .mean()
    .reset_index()
)

plt.figure()

sns.barplot(
    data=tier_data,
    x="review_volume_tier",
    y="approval_rate",
    hue="review_volume_tier",
    palette="viridis",
    legend=False
)

plt.title(
    "Q6: Average Approval Rate by Review-Volume Tier",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Review-Volume Tier")
plt.ylabel("Average Approval Rate (%)")
plt.ylim(0, 100)
plt.show()

low_volume_approval = (
    tier_data.iloc[0]["approval_rate"]
)

high_volume_approval = (
    tier_data.iloc[-1]["approval_rate"]
)

approval_difference = (
    high_volume_approval
    - low_volume_approval
)

display(Markdown(f"""
### Q6 FINDINGS & BUSINESS INSIGHTS

* **Data Point:** Average approval increased from approximately **{low_vo
* **Interpretation:** The difference between the groups was approximately
* **Important Note:** This is a descriptive relationship and does not est
"""))

# =====
# Q7: HELPFUL REVIEW RATE BY GAME

```

```

# =====

helpful_rate_games = game_stats[
    game_stats["total_reviews"] > 10_000
].copy()

top_helpful_rate_games = (
    helpful_rate_games
    .nlargest(
        10,
        "helpful_review_rate_pct"
    )
)

plt.figure()

sns.barplot(
    data=top_helpful_rate_games,
    x="helpful_review_rate_pct",
    y="app_name",
    hue="app_name",
    palette="magma",
    legend=False
)

plt.title(
    "Q7: Top Games by Helpful Review Rate",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Reviews Marked as Helpful (%)")
plt.ylabel("Game")
plt.show()

leading_helpful_game = (
    top_helpful_rate_games.iloc[0]
)

display(Markdown(f"""
Q7 FINDINGS & BUSINESS INSIGHTS

* Data Point: {leading_helpful_game['app_name']} recorded the hig
* Interpretation: Reviews for the leading games were more likely to r
* Business Relevance: These titles may be useful candidates for deepe
"""))

# =====
# Q8: REVIEW VOLUME VS. APPROVAL RATE
# =====

correlation_data = game_stats[
    game_stats["total_reviews"] > 1_000
].copy()

```

```

volume_approval_correlation = (
    correlation_data["total_reviews"]
    .corr(
        correlation_data["approval_rate"]
    )
)

volume_strength, volume_direction = (
    describe_correlation(
        volume_approval_correlation
    )
)

plt.figure()

sns.regplot(
    data=correlation_data,
    x="total_reviews",
    y="approval_rate",
    scatter_kws={
        "alpha": 0.1,
        "color": "teal"
    },
    line_kws={
        "color": "red"
    }
)

plt.xscale("log")

plt.gca().axis.set_major_formatter(
    formatter
)

plt.title(
    "Q8: Review Volume vs. Approval Rate "
    f"(Correlation: {volume_approval_correlation:.3f})",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Total Reviews")
plt.ylabel("Approval Rate (%)")
plt.show()

display(Markdown(f"""
Q8 FINDINGS & BUSINESS INSIGHTS

* Data Point: A {volume_strength} {volume_direction} correlation ({
* Interpretation: Review volume has little meaningful linear relation
* Business Relevance: Review volume alone should not be treated as a
"""))

# =====
# Q9: APPROVAL RATE VS. HELPFUL REVIEW RATE

```

```

# =====

approval_helpful_correlation = (
    correlation_data["approval_rate"]
    .corr(
        correlation_data[
            "helpful_review_rate_pct"
        ]
    )
)

helpful_strength, helpful_direction = (
    describe_correlation(
        approval_helpful_correlation
    )
)

if approval_helpful_correlation < 0:

    helpful_relationship_text = (
        "Games with higher approval rates tended to have "
        "lower helpful-review rates."
    )

elif approval_helpful_correlation > 0:

    helpful_relationship_text = (
        "Games with higher approval rates tended to have "
        "higher helpful-review rates."
    )

else:

    helpful_relationship_text = (
        "No linear relationship was observed."
    )

plt.figure()

sns.regplot(
    data=correlation_data,
    x="approval_rate",
    y="helpful_review_rate_pct",
    scatter_kws={
        "alpha": 0.2
    },
    line_kws={
        "color": "darkorange"
    }
)

plt.title(
    "Q9: Approval Rate vs. Helpful Review Rate "
    f"(Correlation: {approval_helpful_correlation:.3f})",
    fontsize=16,
    fontweight="bold"
)

```

```

plt.xlabel("Approval Rate (%)")
plt.ylabel("Reviews Marked as Helpful (%)")
plt.show()

display(Markdown(f"""
### Q9 FINDINGS & BUSINESS INSIGHTS

* Data Point: A {helpful_strength} {helpful_direction} correlation
* Interpretation: {helpful_relationship_text}
* Important Note: Review usefulness may also be influenced by writing
"""))

# =====
# Q10: CONCENTRATION OF HELPFUL REVIEW ACTIVITY
# =====

total_helpful_reviews = (
    game_stats["helpful_reviews"].sum()
)

top_one_percent_count = max(
    1,
    int(
        np.ceil(
            len(game_stats) * 0.01
        )
    )
)

top_one_percent_games = (
    game_stats
    .nlargest(
        top_one_percent_count,
        "helpful_reviews"
    )
)

top_one_percent_share = (
    top_one_percent_games[
        "helpful_reviews"
    ]
    .sum()
    / total_helpful_reviews
    * 100
)

helpful_concentration_data = pd.DataFrame({
    "Group": [
        "Top 1% of Games",
        "Rest of Market (99%)"
    ],
    "Share": [

```

```

        top_one_percent_share,
        100 - top_one_percent_share
    ]
})

plt.figure(figsize=(10, 6))

sns.barplot(
    data=helpful_concentration_data,
    x="Group",
    y="Share",
    hue="Group",
    palette="flare",
    legend=False
)

plt.title(
    "Q10: Share of Reviews Marked as Helpful",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("")
plt.ylabel("Share of Helpful-Marked Reviews (%)")
plt.show()

display(Markdown(f"""
### Q10 FINDINGS & BUSINESS INSIGHTS

* **Data Point:** The top 1% of games accounted for {top_one_percent_sh
* **Interpretation:** Helpful-review activity is highly concentrated amon
* **Business Relevance:** A limited group of titles generates a dispropor
"""))

# =====
# FINAL EXECUTIVE SUMMARY
# =====

display(Markdown(f"""
# Executive Summary

- The original dataset contained **{initial_records:,} individual review
- After cleaning, **{len(df_clean):,} valid reviews** remained.
- The reviews were aggregated into **{game_stats_count:,} game-level summ
- Each row in `game_stats` represents one unique game name, not one indiv
- Positive reviews represented **{positive_rate:.1f}%** of the analyzed d
- **{not_helpful_percentage:.1f}%** of reviews were not marked as helpful
- The top 5% of games generated approximately **{top_five_share:.1f}% of
- Negative reviews were approximately **{helpful_rate_ratio:.1f} times mo
- Review volume showed a **{volume_strength} {volume_direction} correlati
- Approval rate showed a **{helpful_strength} {helpful_direction} correla
- The top 1% of games accounted for **{top_one_percent_share:.1f}% of all

## Business Interpretation

The analysis shows that review activity and Helpful-mark engagement are c

```

```
Overall user sentiment is positive, but review volume alone does not prov

The findings describe relationships within the dataset and should not be
"""))

print("\n" + "=" * 55)
print("STEAM REVIEW ANALYSIS COMPLETED SUCCESSFULLY")
print("=" * 55)
```

Decoding Steam: A Data-Driven Analysis of 6.4 Million User Reviews

Analysis of review sentiment, market concentration, approval rates, and helpful-review engagement

Tools: Python, Pandas, NumPy, Matplotlib, and Seaborn

Focus: Data quality, review activity, user approval, and helpful-review engagement

Step 1: Loading the Steam dataset...
Steam dataset validated successfully.

```
=====
DATA QUALITY AUDIT REPORT
=====
```

```
Total Records: 6,417,106
Total Columns: 5
```

```
--- Missing Values per Column ---
```

```
app_name      183234
review_text    7305
dtype: int64
```

```
--- Zero Values Analysis ---
```

```
app_id: 0 zeros (0.00%)
review_score: 0 zeros (0.00%)
review_votes: 5,472,222 zeros (85.28%)
```

```
--- Review Votes Validation ---
```

```
Unique review_votes values: [0, 1]
Confirmed: review_votes is a binary field.
0 = the review was not marked as helpful.
1 = the review was marked as helpful.
```

```
=====
DATA CLEANING SUMMARY
=====
```

```
Original Records: 6,417,106
Removed Records: 183,234
Clean Review Records: 6,233,872
```

```
--- Helpful-Review Distribution ---
```

```
Reviews not marked as helpful: 85.34%
Reviews marked as helpful: 14.66%
```

Data Cleaning Explanation

- The original dataset contained **6,417,106 individual review records**.
- **183,234 records** were removed because essential fields were missing or invalid.
- The cleaned dataset contains **6,233,872 valid reviews**.
- **85.34%** of valid reviews were not marked as helpful.
- Zero values were retained because **0** represents a valid review that did not receive a Helpful mark.

The percentage of zero values changed slightly after cleaning because rows with missing or invalid essential fields were removed.

Review-Level Data Preview

Before aggregation, each row represents **one individual Steam review**.

The same game can appear in many different rows because one game may have thousands of user reviews.

	app_id	app_name	review_text	review_score	review_votes	is_helpful
0	10	Counter-Strike	Ruined my life.	1	0	0
1	10	Counter-Strike	This will be more of a "my experience with this game" type of review, beca...	1	1	1
2	10	Counter-Strike	This game saved my virginity.	1	0	0
3	10	Counter-Strike	• Do you like original games? • Do you like games that don't lag? • Do you l...	1	0	0
4	10	Counter-Strike	Easy to learn, hard to master.	1	1	1
5	10	Counter-Strike	No r8 revolver, 10/10 will play again.	1	1	1
6	10	Counter-Strike	Still better than Call of Duty: Ghosts...	1	1	1
7	10	Counter-Strike	cant buy skins, cases, keys, stickers - gaben cant steal our money 10/10 wou...	1	1	1
8	10	Counter-Strike	Counter-Strike: Ok, after 9 years of unlimited fun with friends, I have fina...	1	1	1
9	10	Counter-Strike	Every server is spanish or french. I can now fluently swear in both language...	1	0	0

=====

REVIEW-LEVEL TO GAME-LEVEL AGGREGATION

=====

Clean individual review rows: 6,233,872

Unique game names: 9,363

Rows in the aggregated game_stats table: 9,363

From Individual Reviews to Game-Level Statistics

The cleaned dataset contains **6,233,872 rows**, where each row represents one individual review.

The command `df_clean.groupby("app_name")` groups together all reviews belonging to the same game.

After the aggregation:

- Each row in `game_stats` represents **one game**, not one review.
- The resulting table contains **9,363 game-level rows**.
- In this dataset, this produces **9,363 unique game-name summaries**.
- The original reviews were not deleted. They were summarized into game-level metrics.

Each game-level row contains:

- `total_reviews` — total number of reviews for the game
- `approval_rate` — percentage of positive reviews
- `helpful_reviews` — number of reviews marked as Helpful
- `helpful_review_rate_pct` — percentage of reviews marked as Helpful

Game-Level Table Preview

After aggregation, each row represents **one game**.

The following preview shows how multiple individual reviews were summarized into one game-level row.

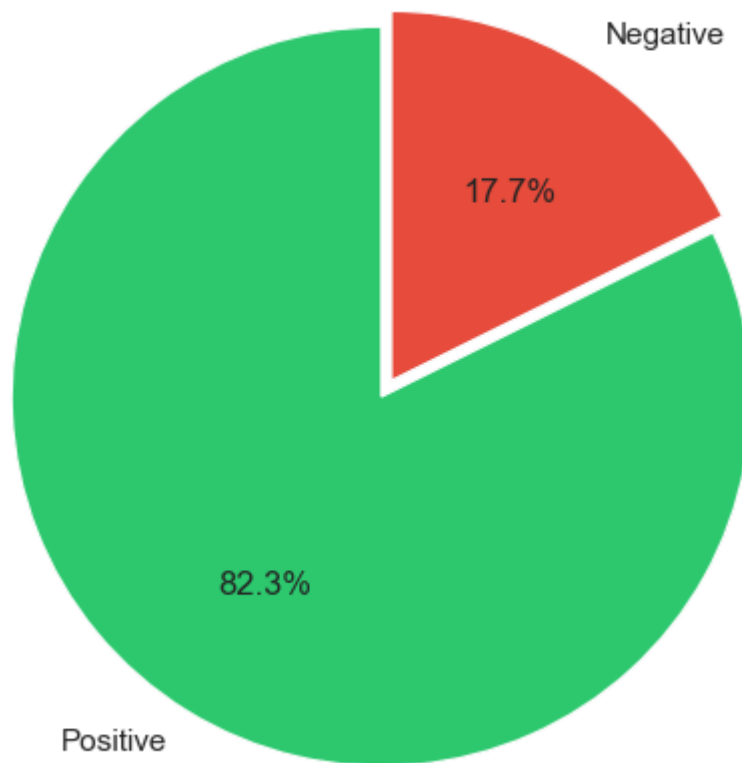
	app_name	total_reviews	approval_rate	helpful_reviews	helpful_review_rate_pct
0	神明的一天世界-God's One Day World	14	78.57	5	35.7
1	! That Bastard Is Trying To Steal Our Gold !	26	30.77	14	53.8
2	!"We Are The Dwarves"!	103	76.70	35	33.9
3	"Glow Ball" - The billiard puzzle game	27	18.52	13	48.1
4	#SelfieTennis	63	74.60	20	31.7
5	#killallzombies	7	71.43	1	14.2
6	\$1 Ride	26	34.62	12	46.1
7	'n Verlore Verstand	19	57.89	11	57.8
8	.EXE	6	83.33	3	50.0
9	001 Game Creator	39	87.18	15	38.4

--- Aggregated Game-Level Statistics ---

	total_reviews	approval_rate	helpful_reviews	helpful_review_rate_pct
count	9,363.00	9,363.00	9,363.00	9,363.00
mean	665.80	72.72	97.63	27.71
std	3,187.72	21.44	449.76	20.37
min	1.00	0.00	0.00	0.00
25%	18.00	60.00	5.00	12.69
50%	58.00	77.78	14.00	23.08
75%	244.00	89.41	46.00	37.50
max	88,973.00	100.00	14,462.00	100.00

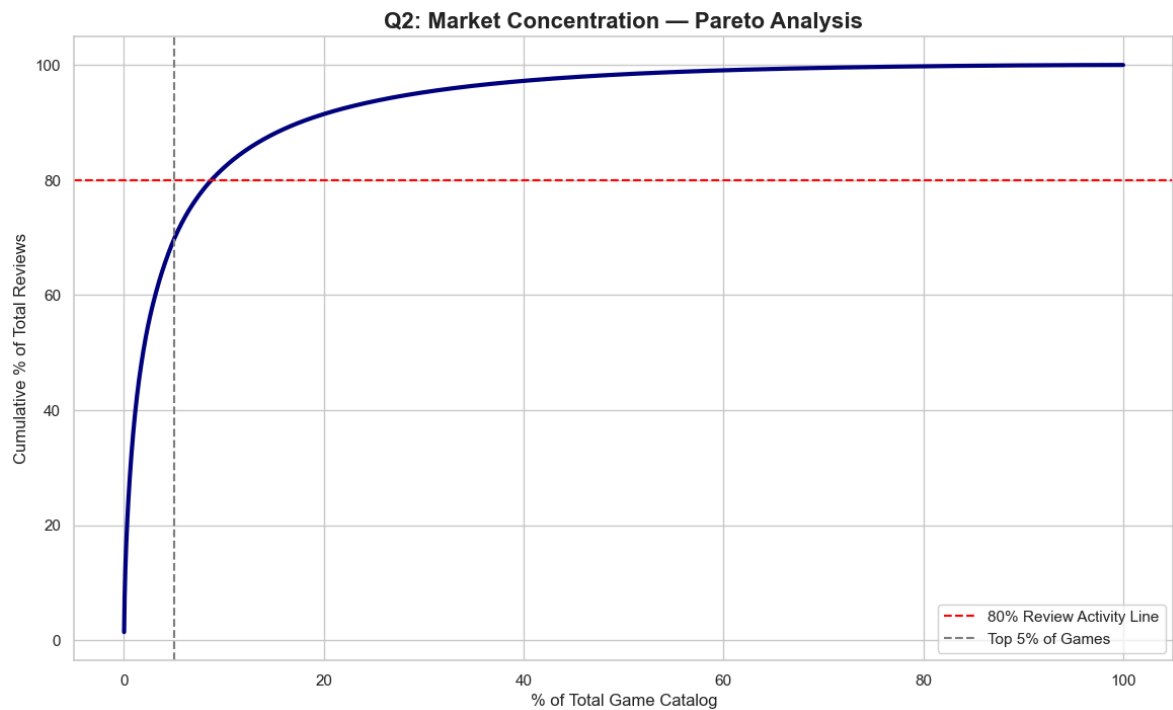
=====

Q1: Global Market Sentiment Benchmark



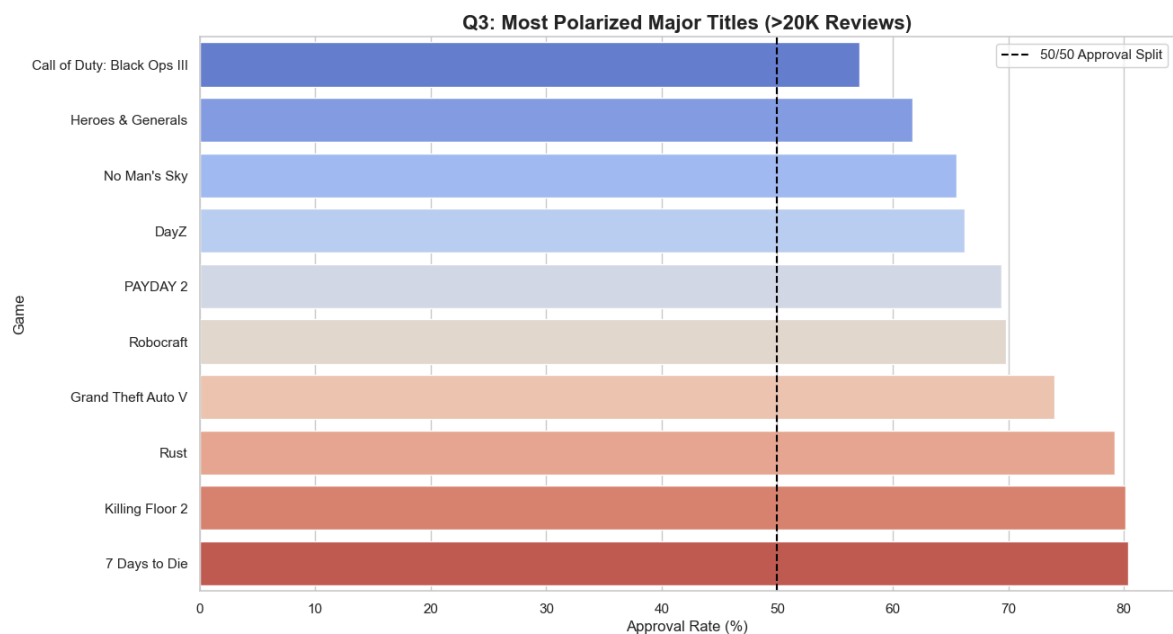
Q1 FINDINGS & BUSINESS INSIGHTS

- **Data Point:** Positive reviews account for **82.3% of the analyzed dataset**.
- **Interpretation:** Overall review sentiment is strongly positive, indicating a high approval benchmark across the analyzed games.
- **Business Relevance:** Games performing substantially below this benchmark may require deeper analysis of product quality, technical issues, or audience fit.



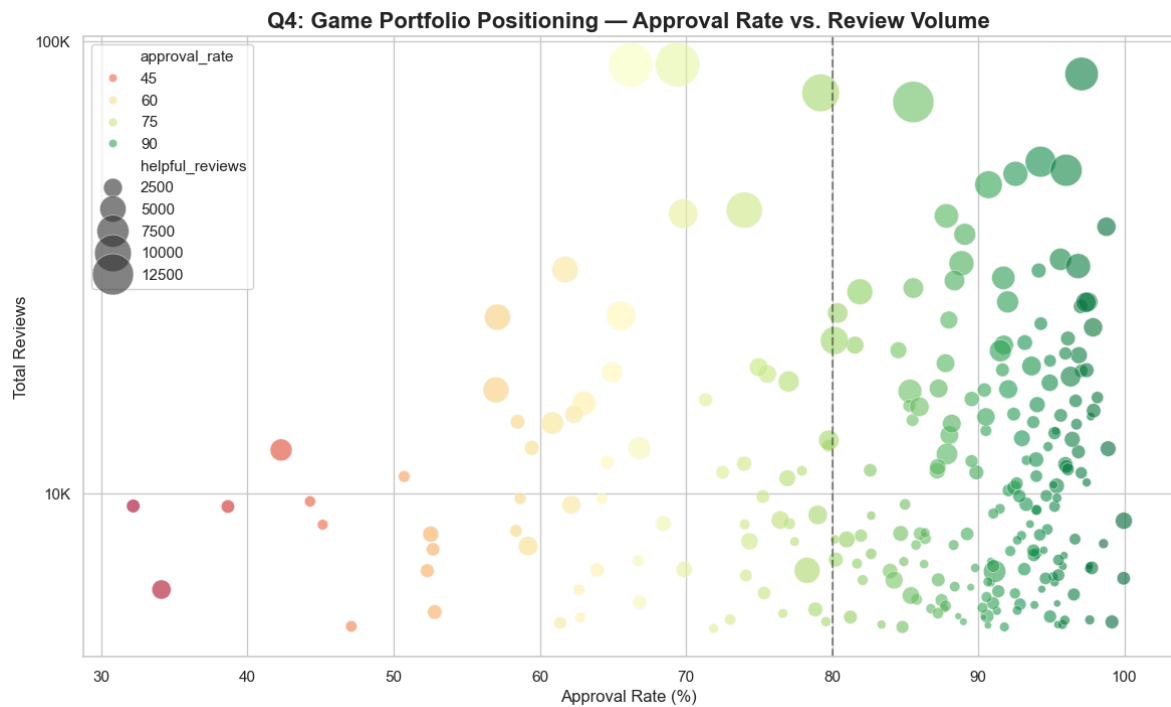
Q2 FINDINGS & BUSINESS INSIGHTS

- **Data Point:** The **top 5% of games** generated **approximately 69.7% of all reviews**.
- **Interpretation:** Review activity is highly concentrated among a relatively small share of games.
- **Business Relevance:** Games outside the leading group may face greater challenges in gaining visibility and generating sustained community activity.



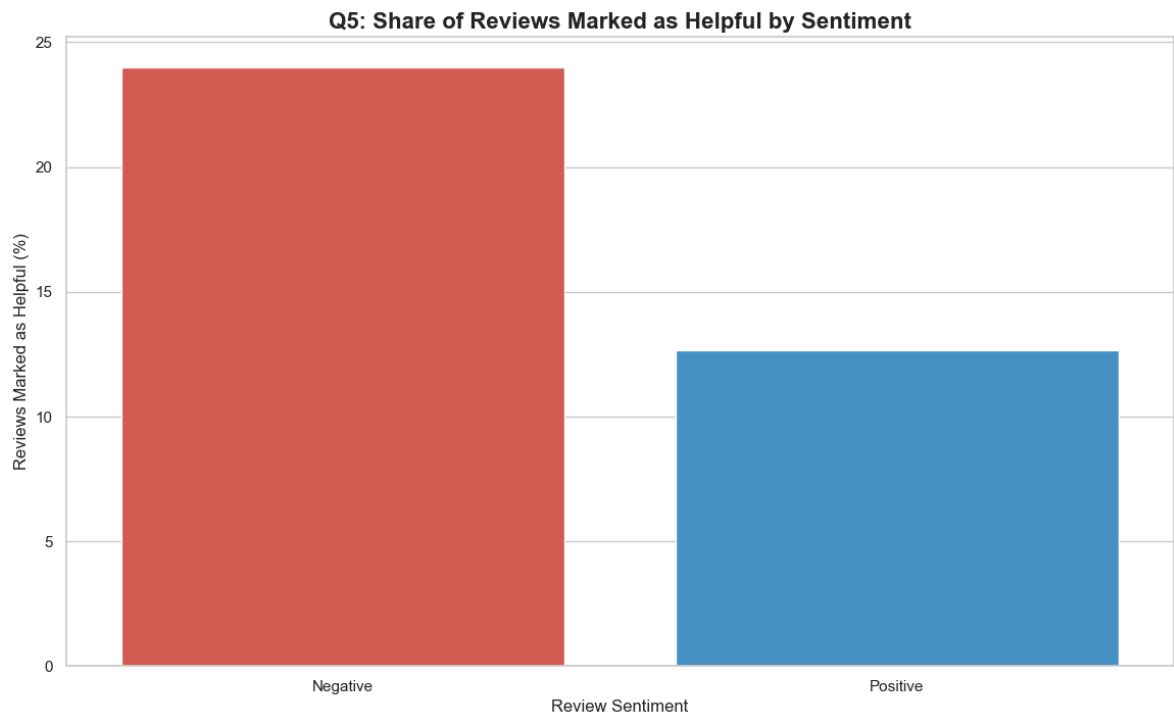
Q3 FINDINGS & BUSINESS INSIGHTS

- **Data Point:** Among games with more than 20K reviews, these titles recorded approval rates closest to a 50/50 split.
- **Interpretation:** The results indicate highly divided user sentiment.
- **Next Step:** Review-text or topic analysis would be required to identify the specific technical, product, or monetization factors behind the divided reception.



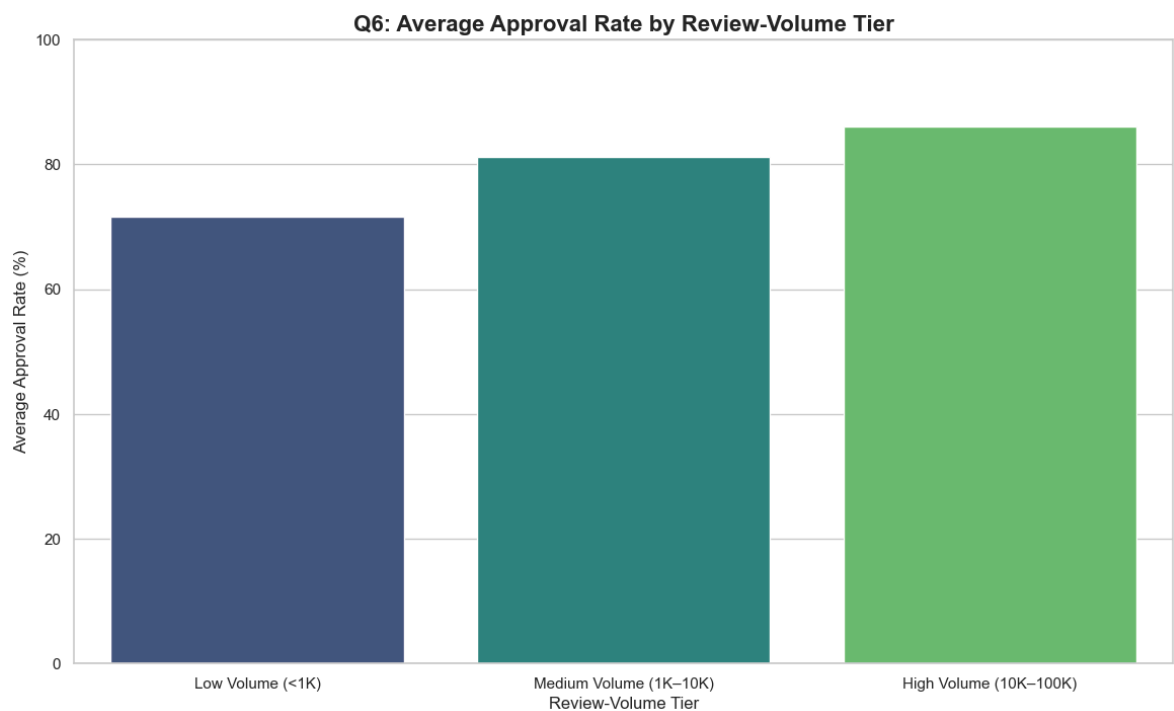
Q4 FINDINGS & BUSINESS INSIGHTS

- **Data Point:** Several games with high review volumes recorded approval rates below the 80% benchmark.
- **Interpretation:** These games combine broad review activity with comparatively weaker user sentiment.
- **Business Relevance:** High-volume, lower-approval games may be prioritized for deeper review analysis and issue identification.



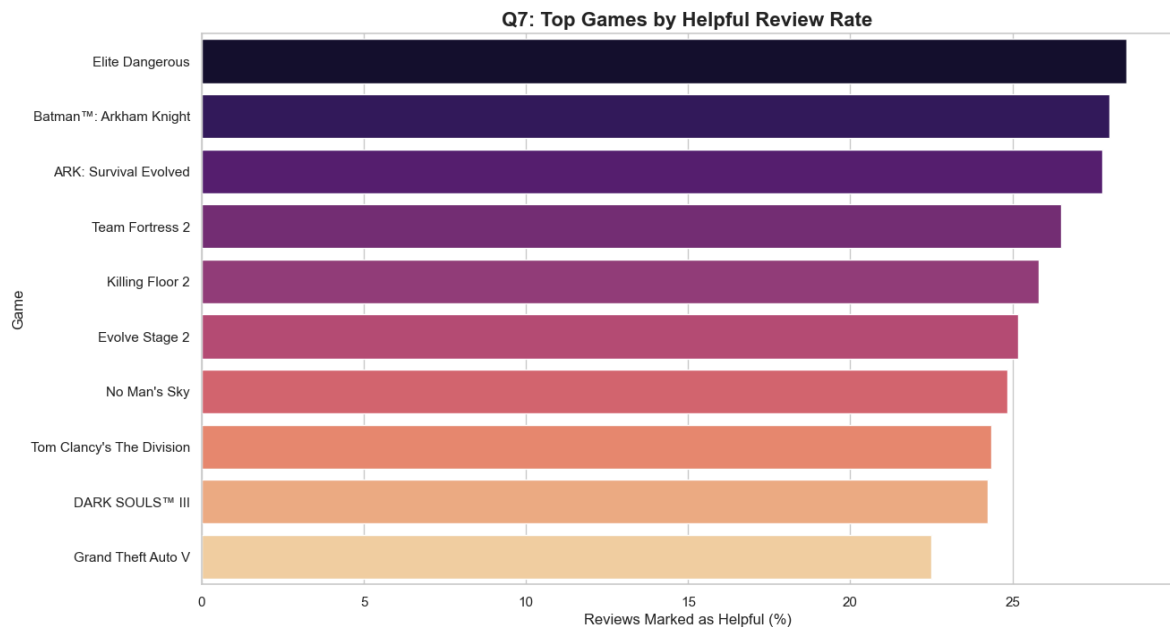
Q5 FINDINGS & BUSINESS INSIGHTS

- **Data Point:** **24.0% of negative reviews** were marked as helpful, compared with **12.7% of positive reviews**.
- **Interpretation:** Negative reviews were approximately **1.9 times more likely** to receive a Helpful mark.
- **Business Relevance:** Critical feedback may receive greater community visibility, making recurring product and technical issues important to monitor.



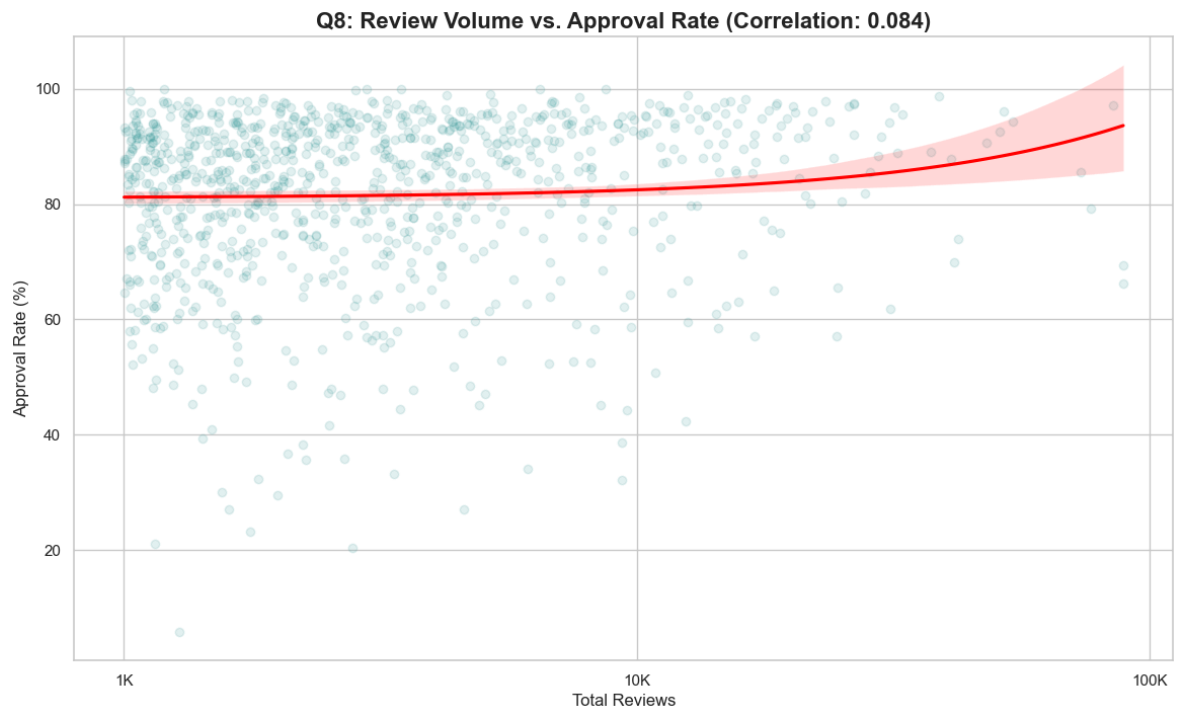
Q6 FINDINGS & BUSINESS INSIGHTS

- **Data Point:** Average approval increased from approximately **71.7%** in the low-volume group to **86.2%** in the high-volume group.
- **Interpretation:** The difference between the groups was approximately **14.5 percentage points**.
- **Important Note:** This is a descriptive relationship and does not establish that higher review volume causes higher user satisfaction.



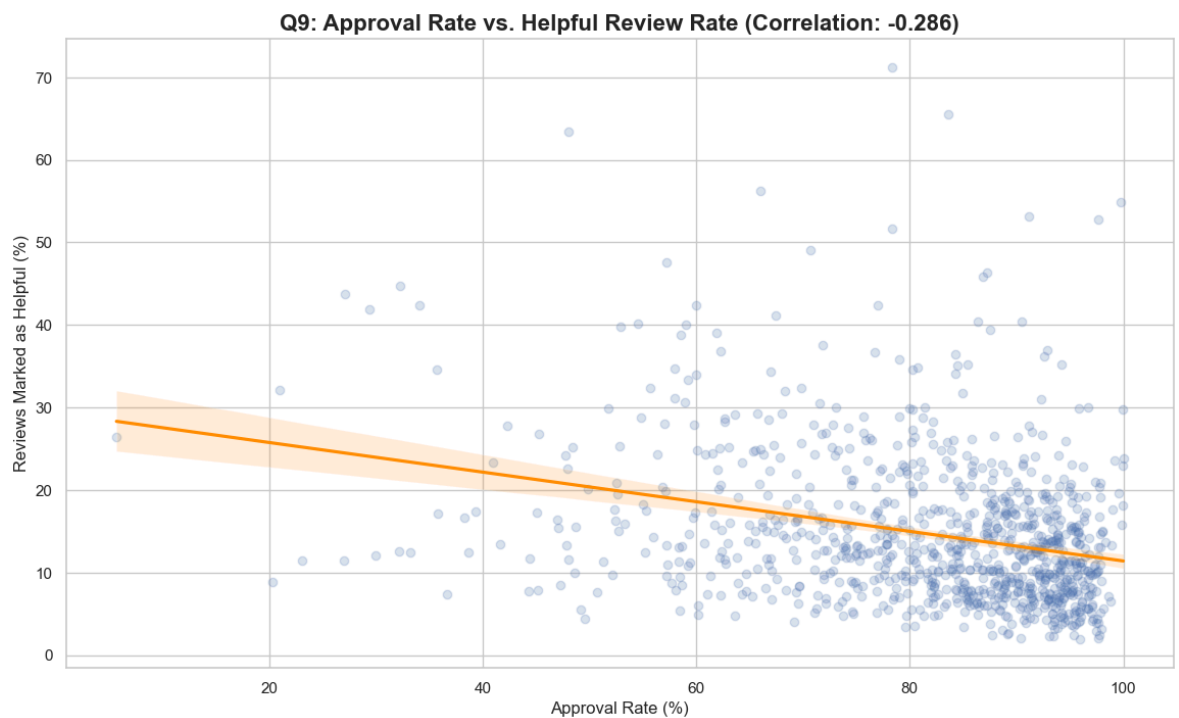
Q7 FINDINGS & BUSINESS INSIGHTS

- **Data Point:** **Elite Dangerous** recorded the highest helpful-review rate at approximately **28.5%**.
- **Interpretation:** Reviews for the leading games were more likely to receive a Helpful mark.
- **Business Relevance:** These titles may be useful candidates for deeper analysis of review quality and community interaction.



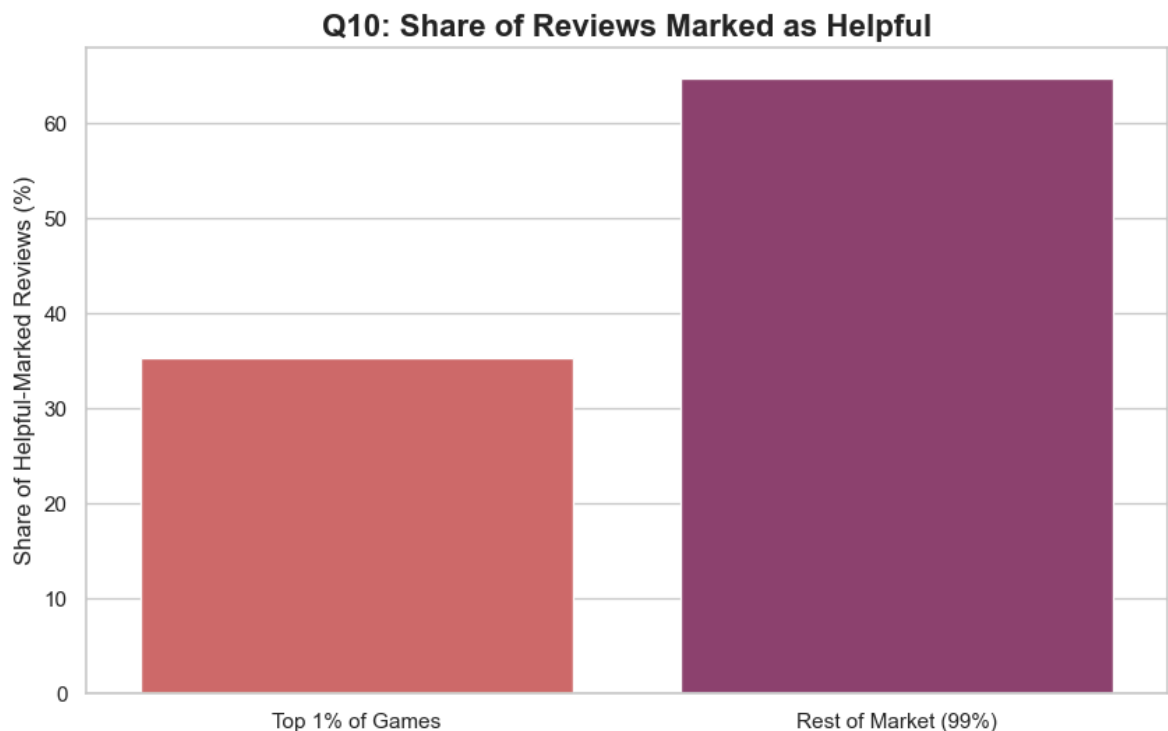
Q8 FINDINGS & BUSINESS INSIGHTS

- **Data Point:** A **very weak positive correlation (0.084)** was observed between review volume and approval rate.
- **Interpretation:** Review volume has little meaningful linear relationship with approval rate in the analyzed sample.
- **Business Relevance:** Review volume alone should not be treated as a reliable indicator of user satisfaction.



Q9 FINDINGS & BUSINESS INSIGHTS

- **Data Point:** A **weak negative correlation (-0.286)** was observed between approval rate and helpful-review rate.
- **Interpretation:** Games with higher approval rates tended to have lower helpful-review rates.
- **Important Note:** Review usefulness may also be influenced by writing quality, timing, topic, and community dynamics. The relationship does not establish causality.



Q10 FINDINGS & BUSINESS INSIGHTS

- **Data Point:** The **top 1% of games accounted for 35.3% of all reviews marked as helpful.**
- **Interpretation:** Helpful-review activity is highly concentrated among a relatively small share of games.
- **Business Relevance:** A limited group of titles generates a disproportionately large share of interaction with review content.

Executive Summary

- The original dataset contained **6,417,106 individual review records**.
- After cleaning, **6,233,872 valid reviews** remained.
- The reviews were aggregated into **9,363 game-level summaries**.
- Each row in `game_stats` represents one unique game name, not one individual review.
- Positive reviews represented **82.3%** of the analyzed dataset.
- **85.3%** of reviews were not marked as helpful, and these valid zero values were retained.
- The top 5% of games generated approximately **69.7% of all reviews**.
- Negative reviews were approximately **1.9 times more likely** to receive a Helpful mark than positive reviews.
- Review volume showed a **very weak positive correlation (0.084)** with approval rate.
- Approval rate showed a **weak negative correlation (-0.286)** with helpful-review rate.
- The top 1% of games accounted for **35.3% of all reviews marked as helpful**.

Business Interpretation

The analysis shows that review activity and Helpful-mark engagement are concentrated among a relatively small share of games.

Overall user sentiment is positive, but review volume alone does not provide a reliable measure of approval. Negative reviews were more likely to receive a Helpful mark, suggesting that critical feedback may gain greater community visibility.

The findings describe relationships within the dataset and should not be interpreted as proof of causality.

```
=====
STEAM REVIEW ANALYSIS COMPLETED SUCCESSFULLY
=====
```

In []: